# Diseño del Prompt Agéntico y Arquitectura de Dominio para la Evaluación Automatizada de NexusMarket en Java

## Fundamentos de Arquitectura de Software y Modelado del Dominio NexusMarket

La plataforma empresarial NexusMarket está concebida como un ecosistema digital centralizado diseñado para la intermediación comercial multifacética entre compradores, vendedores, operadores logísticos, supervisores y administradores. La complejidad técnica del sistema exige la implementación rigurosa de los patrones de Diseño Guiado por el Dominio (Domain-Driven Design o DDD), asegurando que las reglas del negocio gobiernen la estructura del código de forma independiente de las fluctuaciones tecnológicas o de infraestructura.

El modelo operativo del sistema se organiza alrededor de siete dominios funcionales clave, cuya consistencia debe ser preservada mediante límites de agregados claramente delimitados y la observancia incondicional de invariantes del negocio:

1. **Administración de Usuarios:** Constituye el núcleo de identificación y control operacional del sistema. Garantiza la unicidad estricta de credenciales (correo electrónico y documento de identidad) y restringe la asignación de permisos al establecer que cada usuario registrado poseerá un único rol activo dentro de la plataforma.
2. **Gestión de Compradores:** Modela los atributos específicos del cliente comercial, administrando direcciones de entrega principales y secundarias, junto con su estado de habilitación transaccional. Este dominio impone la restricción de que un comprador bajo ninguna circunstancia podrá acceder o modificar la información de otros compradores o inventarios de la plataforma.
3. **Gestión de Vendedores:** Controla la incorporación de proveedores de productos. La regla fundamental de este dominio prohíbe el autorregistro de vendedores, delegando su creación y alta inicial exclusivamente al rol de Administrador.
4. **Gestión de Bodegas:** Administra los espacios físicos de almacenamiento, clasificándolos formalmente entre bodegas pertenecientes al Marketplace y bodegas gestionadas de forma independiente por vendedores.
5. **Gestión del Catálogo:** Clasifica la oferta comercial en productos físicos y digitales. Los productos físicos imponen la gestión obligatoria de inventario distribuido y logística de despacho, mientras que los productos digitales

habilitan la entrega inmediata tras la confirmación del pago. Administra además variantes complejas y estados de publicación acotados.
6. **Gestión del Inventario Distribuido:** Vincula las existencias comerciales a un producto y a una bodega específica. Este dominio aplica la prohibición estricta de existencias negativas e impide la reserva o venta de inventario clasificado bajo el estado de dañado.
7. **Gestión del Ciclo de Vida de Pedidos:** Gobierna el flujo transaccional central del sistema a través de una máquina de estados determinista. Establece que cualquier pedido que haya alcanzado un estado finalizado o entregado es completamente inmutable y no podrá ser modificado bajo ninguna condición.

El desacoplamiento entre el núcleo del negocio y los componentes periféricos de tecnología se logra mediante la adopción de la Arquitectura Hexagonal, conceptualizada formalmente por Alistair Cockburn como el patrón de Puertos y Adaptadores. En esta arquitectura, el centro del sistema está ocupado exclusivamente por las entidades del dominio, los objetos de valor, los servicios de dominio y las interfaces que definen las capacidades operativas del sistema.

El principio fundamental de inversión de dependencias dicta que las capas externas de infraestructura deben depender de las abstracciones definidas por el núcleo, impidiendo que anotaciones de persistencia relacional como `@Entity` de Spring Data JPA o colecciones `@Document` de Spring Data MongoDB contaminen las clases del dominio. La comunicación entre el exterior y el núcleo se canaliza mediante Puertos de Entrada (*Driving / Primary Ports*), que exponen los casos de uso disponibles, y Puertos de Salida (*Driven / Secondary Ports*), que abstraen la persistencia, la mensajería y la integración con servicios de terceros. Los adaptadores traducen las peticiones externas hacia las clases de dominio y viceversa, manteniendo una separación completa de responsabilidades.

# Especificación Estructurada del Prompt Agéntico para Evaluación Automatizada

El prompt de evaluación agéntica se ha estructurado para dirigir a un agente inteligente en el análisis exhaustivo de repositorios estudiantiles Java en la Etapa 1 de modelado DDD. El agente asume el rol de un ingeniero de software experto y evalúa la calidad técnica, el rigor arquitectónico y el cumplimiento de la especificación funcional.

La gestión de la sesión evaluativa se fundamenta en el procesamiento aislado por subcarpeta de estudiante. El agente asigna la identidad del estudiante basándose en el nombre de la carpeta raíz correspondiente (`/{Nombre_Estudiante}/`). Para evitar la alucinación, la deriva cognitiva o la contaminación cruzada de contexto entre revisiones, el agente debe cargar en memoria únicamente el documento de requerimientos funcionales (`especificacion_funcional_nexusmarket.md`), los archivos de especificación documental del estudiante (`/{Nombre_Estudiante}/SDD/`) y el código

fuente Java ubicado en el repositorio (/{Nombre_Estudiante}/src/). Al concluir la generación del informe de evaluación, el agente debe liberar el contexto almacenado antes de proceder con el siguiente repositorio.

El control de flujo operativo incorpora dos mecanismos obligatorios de validación previa:

1. **Verificación de Idempotencia:** El agente debe inspeccionar la raíz del directorio del estudiante para comprobar la existencia previa del archivo evaluacion1.md. Si dicho archivo está presente, el agente omite la evaluación de esa subcarpeta y continúa inmediatamente con el siguiente repositorio.
2. **Regla Descalificante por Lenguaje de Programación:** La implementación debe estar escrita obligatoriamente en lenguaje Java. El agente analiza la estructura de directorios del código fuente y, en caso de no identificar archivos con extensión .java, descalifica el proyecto de manera automática asignando una Nota Final de 0.0 / 5.0. En este escenario, el agente escribe inmediatamente el archivo evaluacion1.md documentando el incumplimiento y finaliza el ciclo para ese repositorio.

La calificación global se calcula a partir de cinco criterios ponderados mediante una función lineal estricta sobre una escala continua de 0.0 a 5.0 puntos. La fórmula cuantitativa para la obtención de la Nota Final se define como:

$$\text{Nota Final} = (C_1 \times 0.20) + (C_2 \times 0.30) + (C_3 \times 0.25) + (C_4 \times 0.15) + (C_5 \times 0.10)$$
Donde la evaluación detallada de los criterios se rige por la escala descriptiva de la siguiente matriz cuantitativa:

| **Criterio Evaluativo** | **Peso** | **Descripción Técnica del Criterio** | **Nivel Excelente (5.0)** | **Nivel Aceptable (3.0 - 3.9)** | **Nivel Insuficiente (1.0 - 2.9)** | **Nivel Pésimo (0.0)** |
|---|---|---|---|---|---|---|
| **Criterio 1: Especificación en SDD** | **20 %** | Cobertura y calidad documental de los modelos del sistema en la | Especificación clara y detallada en /SDD/ que responde rigurosamente a | Especificación presente en /SDD/ con omisiones menores en atributos o reglas | Documentación en /SDD/ extremadamente escasa, confusa o incompleta. | Ausencia total del directorio /SDD/ o archivos de especificación vacíos. |

| Criterio Evaluativo | Peso | Descripción Técnica del Criterio | Nivel Excelente (5.0) | Nivel Aceptable (3.0 - 3.9) | Nivel Insuficiente (1.0 - 2.9) | Nivel Pésimo (0.0) |
|---|---|---|---|---|---|---|
| | | carpeta /SDD/. | los 7 dominios funcionales. | secundarias. | | |
| **Criterio 2: Modelado de Entidades Java** | **30 %** | Definición de entidades de dominio en Java con identidades y comportamiento explícito. | Clases de dominio Java con encapsulamiento estricto, identidades claras y métodos ricos en lógica DDD. | Entidades creadas en Java con mezcla parcial de responsabilidades o acoplamiento a infraestructura. | Modelo anémico basado exclusivamente en getters y setters sin comportamiento del negocio. | Ausencia de entidades del dominio o desorganización total del modelo. |
| **Criterio 3: Value Objects vs. Primitivos** | **25 %** | Sustitución de tipos de datos String abiertos por Enums o Value Objects validados. | Uso consistente de Enums y Value Objects para todos los estados y tipos. Cero String abiertos en campos acotados. | Uso de Enums en los estados principales, pero con persistencia de String en 1 o 2 atributos clave. | Uso masivo de tipos primitivos y String para modelar roles, tipos y estados (obsesión por primitivos). | Inexistencia total de Enums o Value Objects en todo el código Java. |

| Criterio Evaluativo | Peso | Descripción Técnica del Criterio | Nivel Excelente (5.0) | Nivel Aceptable (3.0 - 3.9) | Nivel Insuficiente (1.0 - 2.9) | Nivel Pésimo (0.0) |
|---|---|---|---|---|---|---|
| **Criterio 4: Idioma y Naming en Java** | **15 %** | Adherencia al estándar de codificación Java escrito 100% en idioma inglés. | Código Java escrito completamente en inglés con convenciones de nomenclatura oportunas. | Código Java en inglés en su mayoría, presentando ligeras inconsistencias o términos aislados en español. | Mezcla frecuente de español e inglés (Spanglish) en nombres de clases, métodos y variables. | Código fuente Java escrito totalmente en idioma español. |
| **Criterio 5: Correlación SDD vs. Código** | **10 %** | Trazabilidad y coincidencia biyectiva (1:1) entre la documentación SDD y el código Java. | Correspondencia exactitud 1:1 entre las entidades documentadas en /SDD/ y las clases Java. | Discrepancias menores en nombres de atributos o tipos de datos entre /SDD/ y el código Java. | Poca correspondencia entre los modelos de la documentación y las clases implementadas. | El modelo documentado en /SDD/ no guarda relación alguna con el código Java. |

# Mapeo Métrico del Dominio y Reglas de Invarianza Funcional

Para que el agente evaluador ejecute una inspección precisa del código fuente Java, se definen las matrices formales de coincidencia que vinculan las entidades de negocio con sus correspondientes Value Objects, invariantes operativas, adaptadores e interfaces de puertos.

### Matriz de Agregados, Entidades y Objetos de Valor del Dominio

La verificación del modelado en Java requiere constatar que cada entidad posea atributos obligatorios y tipos acotados que impidan la inconsistencia de datos. La siguiente tabla detalla los componentes estructurales exigidos para cada entidad del dominio NexusMarket:

| **Entidad / Agregado** | **Clasificación DDD** | **Atributos de Dominio Requeridos** | **Enums y Value Objects Obligatorios** | **Reglas de Invarianza y Validación Técnica** |
|---|---|---|---|---|
| **User (Usuario)** | Entidad / Raíz de Agregado | UserId, FullName, Email, UserRole, UserStatus [cite: 1, 5] | UserRole (*Buyer, Seller, LogisticOperator, Admin, Supervisor*)<br><br>UserStatus (*Active, Blocked*) | Identificador y correo electrónico únicos en la plataforma. Cada usuario posee exactamente un único rol dentro del sistema. |
| **Buyer (Comprador)** | Entidad | BuyerId, User, MainAddress, AdditionalAddresses, CommercialStatus [cite: 1, 5] | CommercialStatus (*Eligible, Restricted, Suspended*)<br><br>Address (Value Object) | El comprador no puede administrar inventarios ni acceder a la información privada de otros compradores. |
| **Seller (Vendedor)** | Entidad | SellerId, User, TaxId, Warehouses [cite: 1, 5] | TaxId (Value Object con validación de formato fiscal) | Prohibición absoluta de autorregistro. Su incorporación es realizada exclusivame |

| Entidad / Agregado | Clasificación DDD | Atributos de Dominio Requeridos | Enums y Value Objects Obligatorios | Reglas de Invarianza y Validación Técnica |
|---|---|---|---|---|
| | | | | nte por un usuario con rol Administrador. |
| **Warehouse (Bodega)** | Entidad / Raíz de Agregado | WarehouseId, Name, Location, WarehouseType [cite: 1, 5] | WarehouseType (*Marketplace, Seller*) | Clasificación obligatoria según propiedad. Debe estar asociada a un operador o administrador para la recepción física. |
| **Product (Producto)** | Entidad / Raíz de Agregado | ProductId, Name, Description, ProductType, ProductStatus, Variants [cite: 1, 5] | ProductType (*Physical, Digital*)<br><br>ProductStatus (*Published, Suspended, Discontinued*)<br><br>ProductVariant (Value Object) | Los productos físicos requieren vinculación con el inventario distribuido. Los productos digitales no generan orden de despacho logístico. |
| **Inventory (Inventario)** | Entidad | InventoryId, Product, Warehouse, AvailableQuantit | MovementType (*Inflow, Reservation, Sale,* | Prohibición estricta de existencias negativas |

| Entidad / Agregado | Clasificación DDD | Atributos de Dominio Requeridos | Enums y Value Objects Obligatorios | Reglas de Invarianza y Validación Técnica |
|---|---|---|---|---|
| | | y, ReservedQuantity [cite: 1, 5] | *Adjustment, Return*) | bajo cualquier circunstancia. Impedimento de reserva de stock con estado dañado. |
| **Order (Pedido)** | Entidad / Raíz de Agregado | OrderId, Buyer, CartItems, OrderStatus, TotalAmount, Invoice [cite: 1, 5] | OrderStatus (*Cart, PendingPayment, Paid, Shipped, Delivered*)<br><br>Money (Value Object) | Inmutabilidad total de pedidos finalizados. La transición de estados debe seguir la máquina de estados comercial definida. |

## Matriz de Puertos de Entrada por Rol y Adaptadores Periféricos

La segregación de interfaces en la capa de puertos de entrada (domain/ports/in/) asegura que la autorización y los casos de uso se restrinjan de acuerdo con el rol del usuario autenticado. La capa de adaptadores REST (adapters/rest/) procesa el token JWT, extrae las credenciales del usuario, reconstruye la instancia del agregado User y la transfiere al caso de uso correspondiente para su ejecución.

La tabla a continuación resume la correspondencia entre los puertos de entrada, los roles autorizados y la estructura de adaptadores exigida:

| Puerto de Entrada por Rol (domain/ports/in/) | Roles con Autorización | Casos de Uso y Operaciones Principales | Adaptadores y Mecanismos de Infraestructura |
|---|---|---|---|
| PublicAccessPort | Acceso Público / Todos | Consulta del catálogo público de productos y detalles de variantes. | Controlador REST (/api/v1/public). No requiere validación de cabecera de autenticación JWT. |
| NaturalCustomerPort | Comprador (Buyer) | Gestión del carrito de compras, confirmación de pedidos y solicitudes de reembolso. | Controlador REST (/api/v1/buyer). Valida token JWT con rol Buyer e inyecta la entidad User al caso de uso. |
| BusinessCustomerPort | Vendedor (Seller) | Registro de productos, actualización de variantes y consulta de existencias propias. | Controlador REST (/api/v1/seller). Reconstruye la entidad User con rol Seller desde las claims del JWT. |
| BusinessOperatorPort | Operador Logístico | Despacho de paquetes, registro de movimientos de inventario y recepción de devoluciones. | Controlador REST (/api/v1/logistics). Mapea peticiones al puerto logístico exigiendo rol LogisticOperator. |
| BusinessSupervisorPort | Supervisor | Consulta de métricas operativas, auditoría de | Controlador REST (/api/v1/supervisor). Expone |

| Puerto de Entrada por Rol (domain/ports/in/) | Roles con Autorización | Casos de Uso y Operaciones Principales | Adaptadores y Mecanismos de Infraestructura |
|---|---|---|---|
| | | estados de pedidos y seguimiento. | endpoints de lectura analítica sin capacidad de modificación. |
| TellerEmployeePort | Administrador | Alta de vendedores, asignación de bodegas iniciales y bloqueo de usuarios. | Controlador REST (/api/v1/admin). Ejecuta casos de uso administrativos exigiendo autenticación de Admin. |

La organización de paquetes en el código fuente debe reflejar con claridad la separación de capas de la Arquitectura Hexagonal. El dominio pura de la aplicación se ubica de forma aislada bajo el paquete domain/, conteniendo los submódulos models/ para las entidades Java, enums/ para los Value Objects y tipos acotados, services/ para la lógica de negocio coordinada y ports/ para las interfaces de entrada y salida.

Por su parte, la infraestructura y los detalles técnicos se alojan dentro del paquete adapters/, el cual se subdivide en rest/ para los controladores y DTOs de HTTP, useCases/ para las implementaciones concretas de los puertos de entrada, y persistence/ para las entidades de base de datos, repositorios de Spring Data y mapeadores biyectivos entre el dominio y la persistencia.

| Capa Arquitectónica | Ruta de Paquetes Java | Responsabilidad y Componentes Contenidos |
|---|---|---|
| **Modelo de Dominio** | domain/models/ | Entidades puras y raíces de agregados (User, Product, Order, Inventory) sin anotaciones de frameworks. |
| **Objetos de Valor** | domain/enums/ | Enumeraciones y Value Objects (OrderStatus, UserRole, ProductType, Money) que garantizan invarianzas. |

| Capa Arquitectónica | Ruta de Paquetes Java | Responsabilidad y Componentes Contenidos |
|---|---|---|
| **Servicios de Dominio** | domain/services/ | Lógica de negocio pura que coordina la interacción entre agregados e inyecta los puertos de salida. |
| **Puertos de Entrada** | domain/ports/in/ | Interfaces organizadas por rol de usuario que definen las operaciones permitidas en la plataforma. |
| **Puertos de Salida** | domain/ports/out/ | Interfaces de repositorio que abstraen el acceso a bases de datos relacionales o NoSQL. |
| **Casos de Uso** | adapters/useCases/ | Clases concretas que implementan los puertos de entrada e inyectan los servicios de dominio. |
| **Adaptadores REST** | adapters/rest/ | Controladores HTTP, clases DTO (RequestDTO/ResponseDTO) y mapeadores de tokens JWT. |
| **Persistencia** | adapters/persistence/ | Mapeadores (Domain $\leftrightarrow$ Entity), entidades ORM (@Entity/@Document) y repositorios de Spring Data. |

# Plantillas de Salida para Reportes de Evaluación

### Plantilla del Reporte Individual (evaluacion1.md)

Para cada subcarpeta procesada, el agente evaluador debe escribir la retroalimentación técnica en el archivo {Nombre_Estudiante}/evaluacion1.md, adhiriéndose estrictamente al siguiente formato Markdown:

# Reporte de Evaluación - Etapa 1: Modelado DDD

**Estudiante / Repositorio:** [Nombre_Estudiante]

**Fecha de Evaluación:** [AAAA-MM-DD]

**Nota Final: [Nota global calculada] / 5.0**

# Detalle por Criterios de Evaluación

## 1. Especificación en SDD (20%) - Puntaje: [0.0 - 5.0]

- **Estado:** [Excelente / Aceptable / Insuficiente / Pésimo]
- **Observaciones y Evidencias:**
  - Archivos analizados: [Citar rutas exactas, ej. SDD/Domain/Domain Model.md]
  - Hallazgos respecto a especificacion_funcional_nexusmarket.md: [Detallar la calidad del contenido y la cobertura de dominios]

## 2. Modelado de Entidades en Java (30%) - Puntaje: [0.0 - 5.0]

- **Estado:** [Excelente / Aceptable / Insuficiente / Pésimo]
- **Observaciones y Evidencias:**
  - Clases analizadas: [Citar clases, ej. src/main/java/domain/models/Product.java]
  - Hallazgos: [Detallar encapsulamiento, invarianza de negocio e identidades de las entidades de NexusMarket]

## 3. Value Objects vs. Primitivos (25%) - Puntaje: [0.0 - 5.0]

- **Estado:** [Excelente / Aceptable / Insuficiente / Pésimo]
- **Observaciones y Evidencias:**
  - Enums / Value Objects encontrados: [Citar tipos acotados, ej. src/main/java/domain/enums/OrderStatus.java]
  - Penalizaciones por String abiertos: [Citar atributos que incurren en obsesión por primitivos]

## 4. Idioma y Convenciones en Java (15%) - Puntaje: [0.0 - 5.0]

- **Estado:** [Excelente / Aceptable / Insuficiente / Pésimo]
- **Observaciones y Evidencias:**
  - Evaluación de idioma: [Confirmar si el código Java está 100% en inglés o citar incoherencias de idioma]

## 5. Correlación SDD vs. Código Java (10%) - Puntaje: [0.0 - 5.0]

- **Estado:** [Excelente / Aceptable / Insuficiente / Pésimo]
- **Observaciones y Evidencias:**

    - Nivel de coincidencia: [Explicar la consistencia biyectiva entre la especificación en SDD y la implementación en Java]

## Comentarios Finales y Recomendaciones

- [Resumen cualitativo con fortalezas identificadas y aspectos específicos a refactorizar]

### Plantilla del Reporte Consolidado (notas_finales.md)

Al finalizar la revisión de todos los repositorios del espacio de trabajo, el agente debe generar o actualizar en la raíz el archivo notas_finales.md con la matriz cuantitativa resumida:

# Reporte Consolidado de Evaluaciones - Etapa 1 DDD

**Fecha de Generación:** [AAAA-MM-DD]

**Total Repositorios Evaluados:** [Número de repositorios]

| Estudiante / Carpeta | Estado de Evaluación | Nota Final (0.0 - 5.0) |
|---|---|---|
| Estudiante_Ejemplo_1 | Evaluado exitosamente | 4.8 |
| Estudiante_Ejemplo_2 | Omitido (Ya evaluado previamente) | 3.2 |
| Estudiante_Ejemplo_3 | Descalificado (No implementado en Java) | 0.0 |

## Conclusiones e Implicaciones para la Automatización Evaluativa

La implementación de este esquema de evaluación agéntica sustentado en el diseño de prompt para el dominio NexusMarket demuestra un avance cualitativo en la auditoría automatizada de código. Al acoplar de forma estricta los requerimientos funcionales del negocio con la Rúbrica de Arquitectura Hexagonal y DDD, se garantiza que el software evaluado no solo sea sintácticamente correcto, sino conceptualmente robusto.

La segregación del dominio frente a los detalles de infraestructura mediante Puertos y Adaptadores protege la estabilidad del núcleo transaccional de NexusMarket. Asimismo, el uso de tipos acotados y Value Objects evita las vulnerabilidades típicas provocadas por la obsesión por primitivos, garantizando que el estado del sistema permanezca siempre dentro de los límites válidos del negocio.

La automatización de estas revisiones mediante agentes aislados e idempotentes establece un estándar para la calificación objetiva, trazable e imparcial en entornos de ingeniería de software a gran escala.